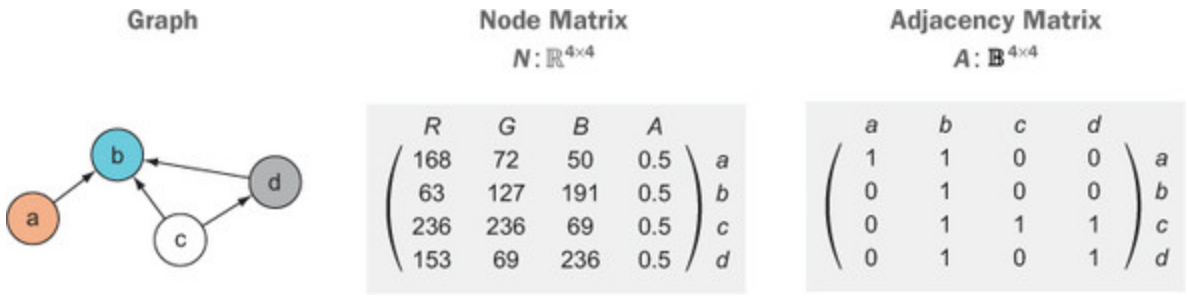
The collection of edges, E, can be represented by an *adjacency matrix*, $E:\mathbb{R}^{n\times n}$, where each row and column are nodes, such that a particular value in row 2, column 3 represents the strength of the relationship between node 2 and node 3 (right panel of figure 10.10). This is the very basic setup for a graph, but graphs can get more complicated where even the edges have feature vectors associated with them. We will not attempt that here.

Figure 10.10. The graph structure on the left can be represented quantitatively with a node feature matrix that encodes the individual node features and an adjacency matrix that encodes the edges (i.e., connections or arrows) between nodes. A 1 in the a row in the b column indicates that node a has an edge from a to b. The node features could be something like an RGBA value if the nodes represented pixels.



A self-attention model works by starting with a set of nodes, $N:\mathbb{R}^{n\times f}$, and then computes the attention weights between all pairs of nodes. In effect, it creates an edge matrix $E:\mathbb{R}^{n\times n}$. After creating the edge matrix, it will update the node features such that each node sort of gets blended together with the other nodes that it attends to. In a sense, each node sends a message to the other nodes to which it most strongly attends, and when nodes receive messages from other nodes, they update themselves. We call this one-step process a *relational module*, after which we get an updated node matrix, $N:\mathbb{R}^{n\times f}$, that we can pass on to another relational module that will do the same thing (figure 10.11). By inspecting the edge matrix, we can see which nodes are attending to which other nodes, and it gives us an idea of the reasoning of the neural network.

Figure 10.11. A relational module, at the highest level, processes a node matrix, $N:\mathbb{R}^{n\times f}$, and outputs a new, updated node matrix, $\hat{N}:\mathbb{R}^{n\times d}$, where the dimensionality of the node feature may be different.